
Data on Fire

A Hands-On Intro to Spark
in Fabric





Thank you to our sponsors!



HI, I'M JASON

THE DAX SHEPHERD*

*More semantic models, fewer sitcoms.



Honored to be
recognized by
Microsoft.

I HERD THE DATA



Microsoft MVP



Nashville, Tennessee



Fabric + Power BI



Teaching + Speaking



MY FAVORITE MODEL

TARDIS

TOOLS & ALLIES



Power BI



Microsoft Fabric



DuckDB



Python



SQL



VS Code

TAKE THIS WITH YOU

DOWNLOAD THIS PRESENTATION

All slides, resources, and demo files
available:

THE DAX SHEPHERD

thedaxshepherd.com/presentations



The DAX Shepherd

thedaxshepherd.com/presentations

Standing on the Shoulders of Giants

Grateful for the people who share their knowledge, encourage us, answer questions, and help the rest of us learn faster.



Photo credit: Unknown Author, CC BY-ND

What We'll Cover

Section 1 / 5

1

Foundations

Spark's origins and the Fabric architecture we'll build on

YOU ARE HERE

2

Python Basics

What you need to know before Spark makes sense

3

PySpark Basics

DataFrames, transforms & actions

4

Power BI from a Notebook

Semantic Link & Semantic Link Labs in action

5

Wrap-Up

Recap, resources & next steps

What We'll Cover

Section 2 / 5

1

Foundations

Spark's origins and the Fabric architecture we'll build on

2

Python Basics

What you need to know before Spark makes sense

YOU ARE HERE

3

PySpark Basics

DataFrames, transforms & actions

4

Power BI from a Notebook

Semantic Link & Semantic Link Labs in action

5

Wrap-Up

Recap, resources & next steps

What We'll Cover

Section 3 / 5

1

Foundations

Spark's origins and the Fabric architecture we'll build on

2

Python Basics

What you need to know before Spark makes sense

3

PySpark Basics

DataFrames, transforms & actions

YOU ARE HERE

4

Power BI from a Notebook

Semantic Link & Semantic Link Labs in action

5

Wrap-Up

Recap, resources & next steps

What We'll Cover

Section 4 / 5

1

Foundations

Spark's origins and the Fabric architecture we'll build on

2

Python Basics

What you need to know before Spark makes sense

3

PySpark Basics

DataFrames, transforms & actions

4

Power BI from a Notebook

Semantic Link & Semantic Link Labs in action

YOU ARE HERE

5

Wrap-Up

Recap, resources & next steps

What We'll Cover

Section 5 / 5

1

Foundations

Spark's origins and the Fabric architecture we'll build on

2

Python Basics

What you need to know before Spark makes sense

3

PySpark Basics

DataFrames, transforms & actions

4

Power BI from a Notebook

Semantic Link & Semantic Link Labs in action

5

Wrap-Up

Recap, resources & next steps

YOU ARE HERE

What We'll Cover

Section 1 / 5

1

Foundations

Spark's origins and the Fabric architecture we'll build on

YOU ARE HERE

2

Python Basics

What you need to know before Spark makes sense

3

PySpark Basics

DataFrames, transforms & actions

4

Power BI from a Notebook

Semantic Link & Semantic Link Labs in action

5

Wrap-Up

Recap, resources & next steps

What lit the fire for **Apache Spark**?

The Netflix Prize

- 1 Began October 2006
- 2 Goal: improve Netflix's Cinematch algorithm by at least 10%
- 3 Prize was \$1 million
- 4 Took until 2009 to win

Couldn't the Elephant* Help?

** Elephant = Hadoop*

Hadoop's Limits

- 1 Hadoop wasn't built for ML — every iteration re-reads from disk
- 2 Iterative algorithms needed a faster execution model
- 3 The shift: keep data in-memory instead of writing to disk each pass
- 4 Like Analysis Services moving from Multidimensional to Tabular

Flashbacks of submitting homework digitally

The 20-Minute Miss

- 1 Front runner: BellKor's Pragmatic Chaos
- 2 A merger of teams from AT&T Labs and Commendo Research
- 3 July 26, 2009 — two teams finally met the minimum requirements
- 4 The Ensemble (Spark) had the better score — but submitted 20 minutes too late

What is Powered by Spark

Microsoft Fabric

Azure Databricks

Azure Synapse Analytics

HDInsight

Amazon EMR · Google Dataproc · Snowflake

Plus thousands of in-house data platforms at Netflix, Uber, Pinterest, Shopify...

UP NEXT

Now let us install Spark



Installing Spark

Step 1 of 42

```
C:\Users\devuser>pip install pyspark
Collecting pyspark
  Downloading pyspark-3.5.1.tar.gz (317.0 MB)
    317.0/317.0 MB 15.2 MB/s 0:00:21
Installing collected packages: py4j, pyspark
Successfully installed py4j-0.10.9.7 pyspark-3.5.1

C:\Users\devuser>python
Python 3.11.6 (tags/v3.11.6:0ba3, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from pyspark.sql import SparkSession
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'pyspark'

C:\Users\devuser>pip show pyspark
WARNING: Package(s) not found: pyspark

C:\Users\devuser>python -m pip install pyspark
Requirement already satisfied: pyspark in c:\users\devuser\appdata\local\programs\python\python311\lib\site-packages (3.5.1)
Requirement already satisfied: py4j==0.10.9.7 in c:\users\devuser\appdata\local\programs\python\python311\lib\site-packages (from pyspark) (0.10.9.7)

C:\Users\devuser>python
>>> from pyspark.sql import SparkSession
>>> spark = SparkSession.builder.appName("test").getOrCreate()
java.lang.IllegalStateException: JAVA_HOME is not set and could not be found in your PATH.
Please set the JAVA_HOME environment variable to point to your Java installation.
    at org.apache.spark.deploy.SparkSubmit$.anonfun$prepareSubmitEnvironment$5 (SparkSubmit.scala:346)
    at org.apache.spark.deploy.SparkSubmit$.submit (SparkSubmit.scala:204)
    at org.apache.spark.deploy.SparkSubmit$.main (SparkSubmit.scala:118)
    at org.apache.spark.deploy.SparkSubmit.main (SparkSubmit.scala)

C:\Users\devuser>set JAVA_HOME=C:\Program Files\Java\jdk-17
C:\Users\devuser>
```

Wait! Microsoft Fabric makes this easy

Running with Scissors

No imports. No SparkSession.builder. No JAVA_HOME.

Just open a Fabric notebook and go.

1 spark

[5] ✓ <1 sec - Command executed in 261 ms by Spark Shepherd on 6/8/2026, 10:25:09 PM

> Log

SparkSession - hive

SparkContext

[Spark UI](#)

Version

v3.5.5.5.4.20260403.6

Master

yarn

AppName

Running_with_Scissors_edd7d87d-a48f-43f5-a649-7101f814b59d

It's already there. Let's use it.

```
1 df = spark.read.table("Legos.sets")
2 df.limit(5).show()
```

[6] ✓ 13 sec - Command executed in 13 sec 414 ms by Spark Shepherd on 6/8/2026, 10:25:22 PM

> Spark jobs (1 of 1 succeeded) Resources Log

set_num	name	year	theme_id	num_parts	img_url
0003977811-1	Ninjago: Book of ...	2022	761	1	https://cdn.rebri...
001-1	Gears	1965	756	43	https://cdn.rebri...
0011-2	Town Mini-Figures	1979	67	12	https://cdn.rebri...
0011-3	Castle 2 for 1 Bo...	1987	199	0	https://cdn.rebri...
0012-1	Space Mini-Figures	1979	143	12	https://cdn.rebri...

Notebooks in Microsoft Fabric

Built for Microsoft Fabric.

The Spark fundamentals you'll learn are portable.

Fabric in Visual Studio Code



Familiar IDE

Stay in the editor your team already uses — no new tool to learn.



GitHub Copilot

AI-assisted authoring for PySpark, SQL, and notebook code.



Notebooks

Author, run, and debug Fabric notebooks with full IDE features.



Fabric compute

Connect to lakehouses and Spark pools running in Microsoft Fabric.

MICROSOFT FABRIC

Architecture at a glance

- Data is stored in **OneLake**
 - ▶ One copy of your data, accessible by every Fabric experience
- **Lakehouse** — two zones
 - ▶ **Files/** unmanaged files (CSV, Parquet, JSON, ...)
 - ▶ **Tables/** managed Delta tables, queryable via SQL endpoint
- Compute reads from OneLake
 - ▶ Spark notebooks, SQL endpoints, Power BI — all hit the same data
- Open formats, no lock-in
 - ▶ Delta Lake + Parquet — readable by any tool that speaks them

Next slide: how all this fits together, visually.

ONELAKE

"THE ONEDRIVE
FOR YOUR DATA"

VISUALLY
EXPLAINED

EACH FABRIC ORGANIZATION HAS
ONE DATA LAKE, ENTIRELY MANAGED FOR YOU
IT'S UNIFIED ACROSS ALL REGIONS & YOU PAY
PER GB STORED (NO SCALING NEEDED).

ALL ANALYTICAL ENGINES CAN ACCESS ONELAKE DATA

SQL

SPARK

POWER BI

KQL DB

DATA LAKE + WAREHOUSE
= LAKEHOUSE

WITH FABRIC, YOUR
WAREHOUSE
DATA IS ALSO STORED
IN ONELAKE, AND
ACCESSIBLE IN AN OPEN
FORMAT

THIS IS A LAKEHOUSE

IT ALLOWS YOU TO DISTRIBUTE
OWNERSHIP TO ORGANIZATIONS,
AND EASILY COLLABORATE ON
DATA

IT CONTAINS

A FILES
AREA

A TABLES
AREA

SHORTCUTS

ALLOW YOU TO QUERY
DATA FROM ONE LAKEHOUSE
TO ANOTHER WITHOUT DATA
MOVEMENT

SHORTCUTS LINK
TO OTHER CLOUDS
TOO!

THE FILES AREA CAN CONTAIN UNSTRUCTURED
& SEMI-STRUCTURED DATA.

IT'S NOT UNCOMMON TO ORGANIZE THEM IN
THREE "AREAS": BRONZE, SILVER & GOLD

ONELAKE

ADLS GEN2
SDK & APIS
CAN NATIVELY ACCESS ONELAKE

Get Noticed in the Fabric Community

- [Browse the Fabric Notebook Gallery](#)
- [First Fabric Notebooks Competition](#)
- [Fabric Semantic Link Developer Expo](#)

1

Browse

Find a notebook in the Gallery that fits a need you have.

2

Build & Share

Solve a pain point and post your notebook to the community.

What We'll Cover

Section 2 / 5

1

Foundations

Spark's origins and the Fabric architecture we'll build on

2

Python Basics

What you need to know before Spark makes sense

YOU ARE HERE

3

PySpark Basics

DataFrames, transforms & actions

4

Power BI from a Notebook

Semantic Link & Semantic Link Labs in action

5

Wrap-Up

Recap, resources & next steps

In Fabric, language choice is also engine choice

Not just syntax — you pick which compute engine runs against OneLake

- **Spark notebook / Lakehouse** — Python, Scala, SparkSQL, or R on the Spark engine
- **Python notebook / Lakehouse** — pure Python, no Spark
- **Warehouse** — T-SQL on the Warehouse SQL engine

Same data. Different language. Different compute engine.

Compute & Language

Spark notebook / Lakehouse

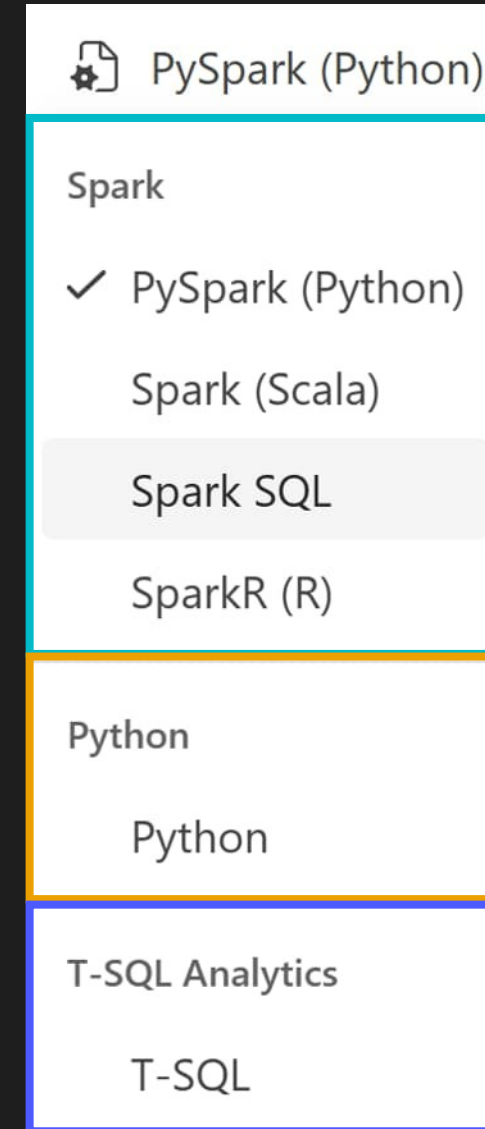
Python, Scala, SparkSQL, R — runs on the Spark engine

Python notebook / Lakehouse

Pure Python — no Spark

Warehouse

T-SQL — runs on the Warehouse SQL engine



Python or PySpark notebook?

Pick Python notebook when...

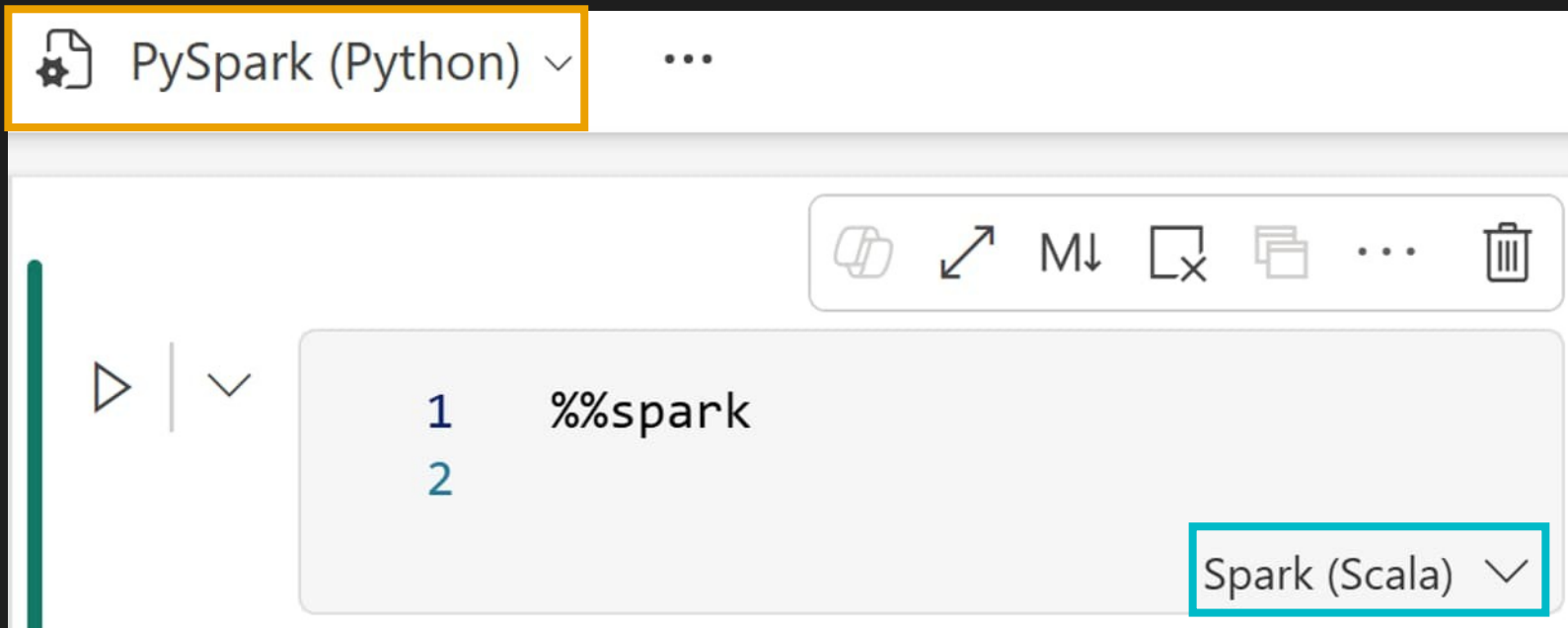
- Data fits in memory (small to medium)
- Rapid exploration & prototyping
- Using DuckDB or Polars (pre-installed)
- Lower cost, faster startup (default: 2 vCores, 16 GB RAM)

Pick PySpark notebook when...

- Large datasets (10GB+) exceeding memory
- Complex ETL pipelines
- High-concurrency or parallel execution
- Need Spark APIs: MLlib, SparkSQL, Streaming

Magic commands — switch language per cell

Notebook language sets the default; cell magics override per cell



Magic command	Runs as
%%pyspark	Python
%%spark	Scala
%%sql	SparkSQL
%%sparkr	R
%%html	HTML

LIVE DEMO

Python Demo



* You're welcome for this being an icon and not an actual snake

What We'll Cover

Section 3 / 5

1

Foundations

Spark's origins and the Fabric architecture we'll build on

2

Python Basics

What you need to know before Spark makes sense

3

PySpark Basics

DataFrames, transforms & actions

YOU ARE HERE

4

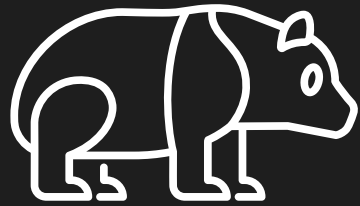
Power BI from a Notebook

Semantic Link & Semantic Link Labs in action

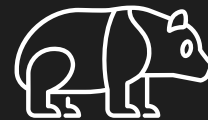
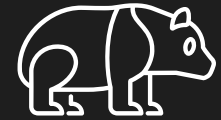
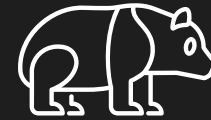
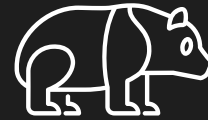
5

Wrap-Up

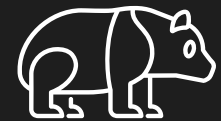
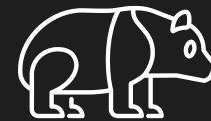
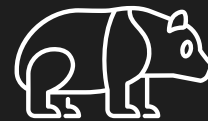
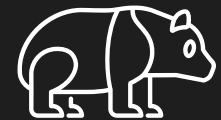
Recap, resources & next steps



Pandas
one machine



**PySpar
k**



across the cluster

Like Pandas, but distributed

PySpark

The Python API for Spark. Write familiar DataFrame code; Spark splits the work across the cluster.

- **Same DataFrame API** — scales from MBs to TBs
 - **Lazy evaluation** — Spark optimizes the full job before running
 - **Native OneLake** — read/write Delta tables directly
 - **SparkSQL on the side** — drop into SQL whenever you want
-

Pandas

```
df = pd.read_parquet("sales.parquet")
(df.groupby("region")
 .agg({"revenue": "sum"}))
```

PySpark — *same shape, runs distributed*

```
df = spark.read.parquet("sales.parquet")
(df.groupBy("region")
 .agg({"revenue": "sum"}))
```

Spark is Lazy
...in a good way



Transformations queue. Actions trigger.

Transformations

Describe the work — deferred

```
select()  
filter()  
withColumn()  
groupBy()  
join()
```

Actions

Demand a result — Spark runs the job

```
show()  
count()  
collect()  
write.save()  
toPandas()
```

WHY LAZY

Benefits of **Lazy Evaluation**

- **Saves resources** — intermediate work is skipped
- **Plan can be optimized** — Catalyst reorders, prunes, pushes down filters
- **Sees the whole job** — Spark picks the smartest path before running anything
- **Skips redundant work** — the same data isn't read or recomputed twice

*For contrast: **pandas** (non-Spark, historic) runs eager evaluation.*

Data Wrangler

Power Query for PySpark
Python



Data Moo-ment

* We're wrangling columns, not cattle.

LIVE DEMO

PySpark Demo



* This is where Python starts speaking Parseltongue to Spark.

What We'll Cover

Section 4 / 5

1

Foundations

Spark's origins and the Fabric architecture we'll build on

2

Python Basics

What you need to know before Spark makes sense

3

PySpark Basics

DataFrames, transforms & actions

4

Power BI from a Notebook

Semantic Link & Semantic Link Labs in action

YOU ARE HERE

5

Wrap-Up

Recap, resources & next steps

Why Semantic Link?

- **The bridge between notebooks and Power BI**
 - Read your model from Python — no export, no REST calls
 - **sempy (Semantic Link)**
 - Datasets, tables, measures, and DAX — as DataFrames
 - **Semantic Link Labs**
 - Extended utilities — best-practice checks, docs, analyzers
-

Explore a Semantic Model

- **List what's in the model**
 - `list_datasets`, `list_tables`, `list_columns`, `list_measures`
 - **Pull a table into a DataFrame**
 - `read_table('FactSales')` — straight into pandas
 - **Visualize relationships**
 - `plot_relationship_metadata()` — see the model topology
-

Evaluate Measures from Python

- **evaluate_measure()**
 - Run any measure without opening Desktop
 - **Slice by any column**
 - Year, Region, Product — same syntax
 - **Returns a DataFrame**
 - Pipe into analysis, charts, or downstream notebooks
-

DAX from a Notebook

- **%%dax cell magic**
 - Write DAX directly in a cell — results render inline
 - **evaluate_dax()**
 - Ad-hoc queries from Python code, parameterized
 - **Results as DataFrames**
 - DAX output flows into pandas / Spark workflows
-

See the Invisible

- **AutoDate hidden tables**
 - Every date column gets a secret table — sempy shows them all
 - **Hidden relationships**
 - Cross-references Power BI Desktop won't display
 - **Referential integrity checks via DAX**
 - Find broken keys and orphan rows before users do
-

Semantic Link Labs

- **Best Practice Analyzer**
 - Run BPA rules against any dataset, straight from Python
 - **Auto-generate documentation**
 - Tables, measures, relationships — Markdown out
 - **Metadata automation**
 - Bulk-edit measures, descriptions, perspectives at scale
-

The Memory Analyzer Payoff

- **Profile your model's footprint**
 - Column-by-column size — see exactly where memory goes
 - **Clean vs AutoDate-bloated**
 - Same data, side by side — the bloat is shocking
 - **Why it matters**
 - Refresh time, query speed, capacity cost — all downstream
-

LIVE DEMO

Semantic Link (Labs) Demo



* Welcome to Kovalsky's Laboratory — where notebooks meet Power BI.

What We'll Cover

Section 5 / 5

1

Foundations

Spark's origins and the Fabric architecture we'll build on

2

Python Basics

What you need to know before Spark makes sense

3

PySpark Basics

DataFrames, transforms & actions

4

Power BI from a Notebook

Semantic Link & Semantic Link Labs in action

5

Wrap-Up

Recap, resources & next steps

YOU ARE HERE

Resources

- Fabric Samples
 - <https://github.com/microsoft/fabric-samples>
 - Semantic Link
 - <https://learn.microsoft.com/en-us/fabric/data-science/semantic-link-overview>
 - Semantic Link Labs
 - <https://github.com/microsoft/semantic-link-labs>
 - Dataset – US Airline On-Time Performance (BTS)
 - <https://www.transtats.bts.gov>
 - 2022-2024 · ~7M flights/year · flights, airlines & airports tables
-

Resources

PySpark Book – Data Analysis with Python and PySpark

- <https://www.oreilly.com/library/view/data-analysis-with/9781617297205/>

PySpark Book – Intro to PySpark (Free HTML version)

- <https://pedropark99.github.io/Introd-pyspark/>

Also in the demo repo

- Notebook 3 — Writing to the Lakehouse (Files/ vs Tables/, write modes, Delta merge, time travel)
 - Notebook 5 — Window Functions in PySpark (ranking, lag/lead, running totals, ntile)
-

Same Python, Bigger Reach

Python Got You Started

pandas on the Lego dataset — familiar code, single machine, instant feedback.

PySpark Raised the Ceiling

The same logic, distributed across a cluster. The data grew — the code barely changed.

Semantic Link Closed the Loop

Notebooks reading and writing your Power BI semantic models — analysis and BI, finally connected.

Thank you



Jason Romans
thedaxshepherd@gmail.com
linkedin.com/in/jasonromans
www.thedaxshepherd.com



Slides + Demos



Session Evaluation